AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, *BS*
Inst., Dept. of Computer Science
National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad
Instructor, Dept. of Computer Science
National University of Computer & Emerging Sciences
160 Industrial Estate, Peshawar, KPK, Pakistan

Version History

1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}

2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

I OPTIMIZATION & LOCAL SEARCH **9**

1 Genetic Algorithms **11**

1.1 What are Genetic Algorithms? . . . 11
1.1.1 The Big Idea . . . 11
1.1.2 Why Use Genetic Algorithms? . . . 11
1.2 Basic Genetic Algorithm . . . 12
1.2.1 Key Terms . . . 12
1.2.2 The Algorithm (Step by Step) . . . 12
1.3 Example: Maximizing 1s in a Binary String . . . 13
1.3.1 Step 1: Initial Population . . . 13
1.3.2 Step 2: Selection . . . 13
1.3.3 Step 3: Crossover and Mutation . . . 14
1.3.4 Step 4: Evaluation . . . 14
1.4 Complete Python Example . . . 14
1.5 Population Initialization (Code Snippet) . . . 16
1.6 Advantages & Disadvantages . . . 16
1.6.1 Advantages . . . 16
1.6.2 Disadvantages . . . 16

Part I

OPTIMIZATION & LOCAL SEARCH

Chapter 1

Genetic Algorithms

1.1 What are Genetic Algorithms?

Genetic Algorithms (GAs) are search-based optimization techniques inspired by **natural selection** and **genetics**. They belong to a broader field called *Evolutionary Computation*.

1.1.1 The Big Idea

Think of it like evolution in nature:

- A **population** of possible solutions (like a group of animals) competes to survive.
- Each solution has a **fitness** score (how good it is).
- Better solutions are more likely to **reproduce** (selection).
- Offspring are created by **combining** two parents (crossover) and sometimes **changing** randomly (mutation).
- Over many **generations**, the population gets better and better.

In plain terms: Instead of trying every possible answer (which can take forever), we start with random guesses, keep the best ones, mix them to create new guesses, add small random changes, and repeat. Over time, we get closer to a good solution.

1.1.2 Why Use Genetic Algorithms?

- **Hard problems:** Many real-world problems (e.g., scheduling, routing) are NP-Hard. Finding the exact best solution can take years. GAs give a "good-enough" answer quickly.
- **No derivatives needed:** Unlike gradient-based methods, GAs only need a way to score solutions. You don't need to compute derivatives.
- **Example:** The Travelling Salesperson Problem (TSP)—finding the shortest route visiting many cities. A fast, near-optimal route is often more useful than waiting for the perfect one.

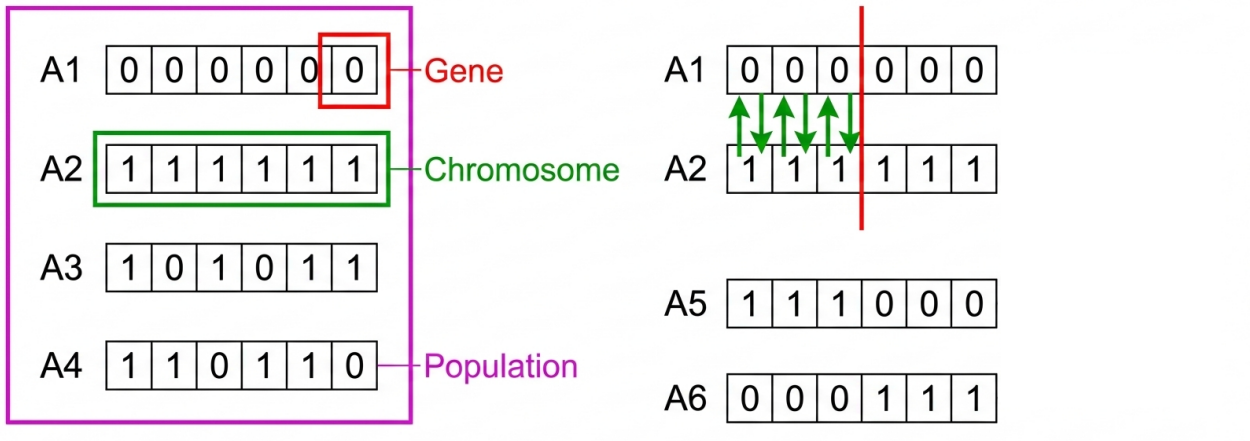
1.2 Basic Genetic Algorithm

1.2.1 Key Terms

- **Chromosome:** One candidate solution (e.g., a binary string, a list of numbers).
- **Population:** A set of chromosomes.
- **Fitness:** A score for each chromosome (higher = better).
- **Selection:** Choosing parents based on fitness.
- **Crossover:** Combining two parents to create offspring.
- **Mutation:** Randomly changing parts of a chromosome.

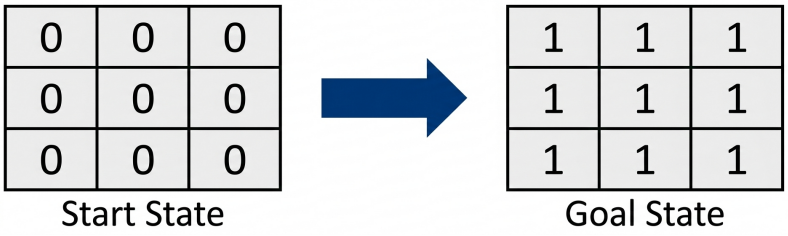
1.2.2 The Algorithm (Step by Step)

1. [**Start**] Create a random population of n chromosomes.
2. [**Fitness**] Evaluate the fitness of each chromosome.
3. [**New population**] Repeat until the new population is full:
   - [**Selection**] Pick two parents (better fitness ⇒ higher chance).
   - [**Crossover**] With some probability, combine parents to form offspring; otherwise copy a parent.
   - [**Mutation**] With a small probability, flip or change genes in the offspring.
   - [**Accept**] Add the offspring to the new population.
4. [**Replace**] Use the new population for the next generation.
5. [**Test**] If a stopping condition is met (e.g., max generations, good enough fitness), stop and return the best solution.
6. [**Loop**] Go back to step 2.



1.3 Example: Maximizing 1s in a Binary String

The Problem: Convert a Start State (3×3 grid of 0s) to a Goal State (3×3 grid of 1s). We represent the grid as a 9-bit string (row by row).



- **Chromosome:** A 9-bit binary string, e.g., `000000001`.
- **Fitness:** The number of 1s in the string. Goal: fitness = 9.

1.3.1 Step 1: Initial Population

We start with random chromosomes:

- P1: `000000001` (Fitness = 1)
- P2: `000000101` (Fitness = 2)
- P3: `000010100` (Fitness = 2)
- P4: `110100100` (Fitness = 4)

1.3.2 Step 2: Selection

Better fitness ⇒ higher chance to be chosen as a parent. A common formula:

$$P(\text{select } i) = \frac{\text{Fitness of chromosome } i}{\text{Sum of all fitness values}}$$

For our example: Total fitness = 1+2+2+4 = 9. So P4 has probability $4/9 \approx 0.44$—almost half the time!

Code explanation: The `fitness` function simply counts 1s using `sum(chromosome)`. The `select_parent` function implements *roulette-wheel selection*: we pick a random number r between 0 and the total fitness, then walk through chromosomes, adding each fitness to a running sum. Whichever chromosome makes the cumulative sum exceed r is selected. This gives fitter chromosomes a proportionally higher chance.

Think of it like a pie chart: each chromosome gets a slice whose size equals its fitness. We spin a pointer to a random position on the pie; whichever slice it lands on is the chosen parent. In code, we build the slices as cumulative ranges: P1 gets $[0, 1)$, P2 gets $[1, 3)$, P3 gets $[3, 5)$, P4 gets $[5, 9)$. A random $r \in [0, 9)$ falls into exactly one of these ranges, so P4 (with the largest slice) is chosen most often.

```
1 import random
2
3 def fitness(chromosome):
4     """Count the number of 1s. Higher is better."""
5     return sum(chromosome)
6
7 def select_parent(population, fitness_scores):
```

```
    """Roulette-wheel selection: pick parent with probability
       proportional to fitness."""
    total = sum(fitness_scores)
    r = random.uniform(0, total)
    cumsum = 0
    for i, f in enumerate(fitness_scores):
        cumsum += f
        if r <= cumsum:
            return population[i]
    return population[-1]
```

Listing 1.1: Fitness function and roulette-wheel selection

1.3.3 Step 3: Crossover and Mutation

- **Crossover:** Combine two parents. Pick a cut point; child gets left part from parent1 and right part from parent2 (and vice versa for child2).
- **Mutation:** Flip each bit with a small probability (e.g., 0.01). This adds diversity and helps escape local optima.

Code explanation: In `crossover`, we split both parents at `point`: child1 gets parent1's left part (`parent1[:point]`) plus parent2's right part (`parent2[point:]`), and child2 gets the opposite. In `mutate`, we loop over each gene: `1 - g` flips a 0 to 1 or a 1 to 0. We only flip when `random.random() < mutation_rate`, so most bits stay unchanged.

```
def crossover(parent1, parent2, point):
    """Single-point crossover: swap tails after point."""
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2

def mutate(chromosome, mutation_rate=0.01):
    """Flip each bit with probability mutation_rate."""
    return [1 - g if random.random() < mutation_rate else g
            for g in chromosome]
```

Listing 1.2: Single-point crossover and mutation

1.3.4 Step 4: Evaluation

Compute fitness of new offspring, check if we reached the goal (fitness = 9), and replace the old population with the new one.

1.4 Complete Python Example

Here is a full, runnable Genetic Algorithm for the "maximize 1s" problem:

Code walkthrough:

- **Constants:** `CHROM_LEN=9`, `POP_SIZE=20`, `GENERATIONS=50`. `MUTATION_RATE=0.05` means each bit has 5% chance to flip; `CROSSOVER_RATE=0.8` means 80% of the time we crossover, 20% we copy parents.
- **Main loop:** Each generation, we compute fitness for all chromosomes, find the best one, and print it. If best fitness equals 9, we stop.

- **Building new population:** We repeatedly select two parents (using roulette-wheel), optionally crossover them to get two children, mutate both children, and add them to `new_pop`. We trim to `POP_SIZE` in case we added one extra pair.

```python
import random

CHROM_LEN = 9
POP_SIZE = 20
GENERATIONS = 50
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8

def create_random_chromosome():
    return [random.randint(0, 1) for _ in range(CHROM_LEN)]

def fitness(chromosome):
    return sum(chromosome)

def select_parent(population, fitness_scores):
    total = sum(fitness_scores)
    r = random.uniform(0, total)
    cumsum = 0
    for i, f in enumerate(fitness_scores):
        cumsum += f
        if r <= cumsum:
            return population[i]
    return population[-1]

def crossover(p1, p2):
    pt = random.randint(1, CHROM_LEN - 1)
    c1 = p1[:pt] + p2[pt:]
    c2 = p2[:pt] + p1[pt:]
    return c1, c2

def mutate(chromosome):
    return [1 - g if random.random() < MUTATION_RATE else g
            for g in chromosome]

# Main GA loop
population = [create_random_chromosome() for _ in range(POP_SIZE)]

for gen in range(GENERATIONS):
    scores = [fitness(c) for c in population]
    best_idx = max(range(len(scores)), key=lambda i: scores[i])
    best_fit = scores[best_idx]
    print(f"Gen {gen}: best = {best_fit}, chrom = {population[best_idx]}")

    if best_fit == CHROM_LEN:  # Goal reached
        print("Goal reached!")
        break

    new_pop = []
    while len(new_pop) < POP_SIZE:
        p1 = select_parent(population, scores)
        p2 = select_parent(population, scores)
        if random.random() < CROSSOVER_RATE:
            c1, c2 = crossover(p1, p2)
        else:
            c1, c2 = p1[:], p2[:]
        c1 = mutate(c1)
```

```
        c2 = mutate(c2)
        new_pop.extend([c1, c2])
    population = new_pop[:POP_SIZE]
```

Listing 1.3: Complete GA: maximize 1s in a 9-bit string

Output (typical run): You will see the best fitness increase over generations until it reaches 9 (all 1s).

1.5 Population Initialization (Code Snippet)

Code explanation: The function uses a nested list comprehension. The inner part `[random.randint(0` `for _ in range(chrom_length)]` creates one chromosome (a list of random 0s and 1s). The outer `for _ in range(size)` repeats this `size` times. Each chromosome is independent and random.

```
def create_population(size, chrom_length):
    """Create a population of random binary chromosomes."""
    return [[random.randint(0, 1) for _ in range(chrom_length)]
            for _ in range(size)]

# Example: 10 chromosomes, each 9 bits long
pop = create_population(10, 9)
print(pop[0])  # e.g., [1, 0, 0, 1, 1, 0, 0, 1, 0]
```

Listing 1.4: Creating the initial population

1.6 Advantages & Disadvantages

1.6.1 Advantages

- **Modular:** The GA framework is separate from your problem. You only need to define chromosome representation and fitness.
- **Improves over time:** Each generation tends to be better than the previous one.
- **Parallel-friendly:** Fitness evaluation can be done in parallel.

1.6.2 Disadvantages

- **Computational cost:** Evaluating fitness (e.g., training a model) can be expensive.
- **Stochastic:** Results vary between runs; may take many generations to converge.
- **Tuning:** You need to choose population size, mutation rate, crossover rate, etc.

Bibliography

[1] Agents in artificial intelligence.

[2] Artificial intelligence.

[3] Artificial intelligence.

[4] Artificial intelligence - agents and environments.

[5] Cs6-1 lab 06.

[6] Networkx - python software package for complex networks.

[7] Types of artificial intelligence.

[8] What is intelligent agent in ai - types and function.